

Introdução ao JavaScript

Do zero até ler, decidir e calcular: os fundamentos da linguagem explicados linha por linha, com exemplos nos temas lanche, moda e viagem.

Material de apoio para quem está construindo o próprio PWA.
Leia com o navegador aberto ao lado e teste cada exemplo.

SUMÁRIO

O que tem neste guia

Pode ler em ordem ou pular direto para a dúvida do momento.

- 01 O que é JavaScript e onde ele roda
- 02 Variáveis: let e const
- 03 Tipos de dado: string, number, boolean
- 04 Operadores: contas e comparações
- 05 Encontrando elementos: document.getElementById
- 06 Lendo e mudando texto: textContent
- 07 Reagindo a eventos: addEventListener
- 08 Lendo formulários: value, checked e Number()
- 09 Decisões: if, else if e else
- 10 Funções: organizando o código
- 11 Tudo junto: um projeto completo, linha por linha
- 12 Erros comuns e como ler o Console
- 13 Glossário rápido

SOBRE ESTE GUIA

Ele não substitui a aula, é um apoio para consultar depois. Os exemplos usam os três temas mais comuns na turma (lanche, roupa e viagem), mas a lógica é a mesma para qualquer projeto. Onde aparece um trecho de código, tente digitar (não copiar e colar) e rodar no seu próprio arquivo antes de seguir para o próximo.

01 • FUNDAMENTOS

O que é JavaScript e onde ele roda

Antes de escrever qualquer código, vale entender o que essa linguagem faz e onde ela vive dentro da página.

HTML dá a estrutura de uma página (títulos, botões, campos de formulário). CSS dá a aparência (cores, espaçamento, fontes). JavaScript dá o comportamento: o que acontece quando alguém clica em algo, digita algo ou marca uma opção. É a única das três linguagens que consegue tomar decisões e reagir a ações do usuário enquanto a página está aberta.

Um jeito simples de pensar nisso: HTML é o esqueleto, CSS é a roupa, JavaScript é o que faz a pessoa se mexer.

Onde o código JavaScript mora

Na maioria dos projetos desta turma, o JavaScript fica em um arquivo separado, geralmente chamado `script.js`, e é ligado ao HTML com uma tag `<script>` perto do fim do `<body>`:

`index.html`

```
<body>
  <h1>Monte seu lanche</h1>
  <!-- todo o conteúdo da página fica aqui -->

  <script src="script.js"></script>
</body>
```

POR QUE NO FIM DO BODY

O navegador lê o HTML de cima para baixo. Se o `<script>` vier antes dos elementos que ele usa, o JavaScript tenta encontrar um botão ou um campo que ainda não existe na página e o código quebra. Colocando o script por último, todo o HTML já foi lido quando o JavaScript começa a rodar.

O Console: onde você vê o que está acontecendo

Todo navegador tem uma ferramenta de desenvolvedor embutida. Para abrir, aperte **F12** (ou botão direito na página → Inspecionar) e clique na aba **Console**. É ali que aparecem os erros do seu código e é ali que você pode imprimir valores para conferir se o JavaScript está lendo o que você espera:

```
console.log("Cheguei aqui");
console.log(precoTotal);
```

`console.log(...)` imprime qualquer coisa entre parênteses no Console. Não aparece na página, é só para você enquanto está construindo e testando. É a ferramenta mais usada para descobrir por que algo não está funcionando.

HÁBITO ÚTIL

Sempre que uma parte do código não fizer o que você espera, o primeiro passo é colocar um `console.log` logo antes ou depois da linha suspeita. Se o valor impresso já estiver errado, o problema está antes daquele ponto. Se estiver certo, o problema está depois.

Variáveis: let e const

Uma variável é uma caixa com nome onde o JavaScript guarda um valor para usar depois.

Sempre que o código precisa lembrar de alguma coisa (um preço, um nome, se uma opção está marcada), esse valor é guardado em uma variável. Em vez de escrever o mesmo valor várias vezes no código, você guarda ele uma vez com um nome e usa esse nome depois.

```
let precoLanche = 15;  
const nomeDaLoja = "Lanchonete da Ana";
```

código	o que significa
let	palavra que diz "estou criando uma variável que pode mudar de valor depois".
precoLanche	o nome que você escolheu para essa variável. É esse nome que você vai usar no resto do código para se referir a esse valor.
= 15	o sinal de igual aqui não é "igual" como em matemática, é "guarda este valor". A variável precoLanche passa a valer 15.
;	ponto e vírgula marca o fim da instrução. É como o ponto final de uma frase em português.
const	igual ao let, mas diz "este valor não vai mudar depois de criado". Tentar mudar dá erro.

Quando usar let e quando usar const

A regra prática usada nesta turma: comece sempre pensando em `const`. Se em algum momento do código você precisar atribuir um novo valor para aquela variável (por exemplo, um total que é recalculado toda vez que o usuário muda uma opção), aí sim use `let`.

```
const nomeDoProduto = "Camiseta Estampada"; // nunca muda depois de criado  
let total = 0; // vai mudar toda vez que o usuário  
escolher algo
```

ERRO COMUM

`const total = 0;` e depois, mais adiante no código, `total = total + 15;` vai dar erro ("Assignment to constant variable"). Se a variável vai ser recalculada, ela precisa nascer como `let`.

Regras de nome

- Não pode começar com número (`1preco` não funciona, `preco1` funciona).
- Não pode ter espaço. Quando o nome tem mais de uma palavra, a convenção usada nesta turma é começar a segunda palavra com maiúscula: `precoTotal`, `nomeDoProduto`, `extraBacon`.
- Não use acentos nem `ç` em nomes de variável. `tamanhoGrande` sim, `tamanhoGrandé` não.
- O nome deve dizer o que a variável guarda. `precoTotal` é melhor que `x` ou `valor2`, mesmo que os três funcionem.

Exemplos nos três temas

```
// lanche
let tamanhoEscolhido = "Duplo";
let precoBase = 20;

// roupa
let corEscolhida = "Azul";
let precoCamiseta = 39;

// viagem
let classeEscolhida = "Executiva";
let precoPassagem = 900;
```

Tipos de dado: string, number, boolean

Todo valor em JavaScript tem um tipo. Os três que você vai usar o tempo todo são estes.

string: texto

Qualquer texto entre aspas é uma **string**. Pode usar aspas simples `'assim'` ou duplas `"assim"`, as duas funcionam igual. O importante é ser consistente.

```
let nomeProduto = "Camiseta Estampada";  
let corEscolhida = 'Azul';
```

number: número

Números não levam aspas. Se colocar aspas, deixa de ser número e vira string, mesmo parecendo um número.

```
let precoBase = 39;           // number: dá para somar, multiplicar  
let precoErrado = "39";      // string: JavaScript trata como texto, não como conta
```

O ERRO MAIS COMUM DO CURSO

Todo valor que vem de um campo de formulário (`.value`) chega como **string**, mesmo que pareça um número. `"15" + "20"` não vira 35, vira o texto `"1520"`, porque o `+` entre strings gruda os textos em vez de somar. A seção 08 explica como resolver isso com `Number()`.

boolean: verdadeiro ou falso

Um **boolean** só pode valer `true` (verdadeiro) ou `false` (falso). Não leva aspas. É o tipo que aparece toda vez que você pergunta "isso está marcado?" ou "essa condição é verdadeira?".

```
let aceitouTermos = true;  
let temDesconto = false;
```

Conferindo o tipo de um valor

Se quiser confirmar o tipo de alguma coisa enquanto testa o código, use `typeof`:

```
console.log(typeof 39);           // "number"
console.log(typeof "39");         // "string"
console.log(typeof true);         // "boolean"
```

TESTE VOCÊ MESMO

Abra o Console (F12) em qualquer página e digite `typeof 10 + 5`. Depois digite `typeof (10 + 5)`. Os parênteses mudam o resultado, porque mudam o que o `typeof` está de fato medindo. Vale testar para entender na prática, não só ler.

Operadores: contas e comparações

Os símbolos que fazem o JavaScript calcular e comparar valores.

Operadores aritméticos

operador	o que faz
+	soma números. Entre strings, gruda os textos (concatenação).
-	subtração.
*	multiplicação.
/	divisão.

```
let subtotal = 39 + 15;           // 54
let comDesconto = subtotal - 5;   // 49
let dobro = subtotal * 2;         // 108
```

Concatenando texto com valores

O mesmo `+` que soma números também gruda strings. É assim que se monta uma frase que mistura texto fixo com um valor calculado, como no `textContent` que a seção 06 explica:

```
let total = 54;
let mensagem = "Total: R$ " + total;
console.log(mensagem); // "Total: R$ 54"
```

Se `total` tivesse casas decimais demais (por exemplo `53.999999` por causa de uma conta com vírgula), use `.toFixed(2)` para arredondar para duas casas antes de montar a frase:

```
let mensagem = "Total: R$ " + total.toFixed(2); // sempre duas casas, tipo "53.99"
```

Operadores de comparação

Comparações não fazem conta, elas perguntam algo e respondem com um **boolean** (`true` ou `false`). São a base de todo `if`, que a seção 09 explica em detalhe.

operador	pergunta que ele faz
===	os dois valores são exatamente iguais (mesmo valor e mesmo tipo)?
!==	os dois valores são diferentes?
>	o primeiro é maior que o segundo?
<	o primeiro é menor que o segundo?
>=	o primeiro é maior ou igual ao segundo?
<=	o primeiro é menor ou igual ao segundo?

```
console.log(10 === 10);    // true
console.log(10 === "10"); // false, número e string não são o mesmo tipo
console.log(total >= 100); // true se total for 100 ou mais
```

= não é o mesmo que ===

Um sinal de igual (=) guarda um valor numa variável. Três sinais de igual (===) perguntam se dois valores são iguais. Confundir os dois é um dos erros mais comuns em quem está começando, e o navegador raramente avisa: o código roda, só que fazendo a coisa errada.

Operadores lógicos: combinando condições

&& significa "e" (as duas coisas precisam ser verdadeiras). || significa "ou" (pelo menos uma precisa ser verdadeira).

```
console.log(total > 50 && total < 200); // true só se as duas forem verdadeiras
console.log(corEscolhida === "Azul" || corEscolhida === "Preto"); // true se for uma das duas
```

Encontrando elementos: document.getElementById

O primeiro passo de quase todo código JavaScript nesta turma: achar o elemento HTML que você quer controlar.

O JavaScript enxerga a página HTML como uma árvore de elementos, chamada **DOM** (Document Object Model). Para o JavaScript conseguir mudar um título, ler um campo de formulário ou reagir a um clique, ele primeiro precisa achar aquele elemento específico dentro da árvore. É isso que

`document.getElementById` faz.

index.html

```
<p id="textoResultado">Escolha uma opção</p>
```

script.js

```
const textoResultado = document.getElementById('textoResultado');
```

código

o que significa

`const textoResultado`

você está criando uma variável para guardar o elemento encontrado. O nome pode ser qualquer um, mas usar o mesmo nome do id ajuda a não se perder.

`document`

representa a página HTML inteira. Todo comando que mexe na página parte daqui.

`.getElementById(...)`

método que procura, em toda a página, um elemento com um id específico.

`'textoResultado'`

o id que você está procurando, escrito exatamente igual ao id do HTML, entre aspas.

O ERRO QUE MAIS APARECE NO CONSOLE

Se o id escrito no JavaScript não bater letra por letra (maiúscula, minúscula, espaço) com o id no HTML, `getElementById` não encontra nada e devolve `null`. Qualquer linha depois disso que tente usar essa variável (`textoResultado.textContent = ...`) quebra com um erro do tipo *"Cannot set properties of null"*. Sempre que aparecer esse erro, o primeiro lugar para olhar é se o id está escrito igual nos dois arquivos.

Um id por elemento

Cada `id` só pode aparecer uma vez em toda a página. Se dois elementos tiverem o mesmo id, `getElementById` vai sempre pegar o primeiro que encontrar e ignorar o resto, o que costuma causar bugs difíceis de perceber. Se for algo que se repete (como vários itens de uma lista), o id de cada um precisa ser diferente: `item1`, `item2`, e assim por diante.

Buscando os elementos de cada tema

```
// lanche
const tamanhoGrande = document.getElementById('tamanhoGrande');
const extraBacon = document.getElementById('extraBacon');

// roupa
const tamanhoP = document.getElementById('tamanhoP');
const estampaPersonalizada = document.getElementById('estampaPersonalizada');

// viagem
const classeEconomica = document.getElementById('classeEconomica');
const extraBagagem = document.getElementById('extraBagagem');
```

Lendo e mudando texto: textContent

Depois de achar um elemento, `textContent` é como você lê ou escreve o texto dentro dele.

index.html

```
<p id="textoResultado">Escolha uma opção</p>
```

script.js

```
const textoResultado = document.getElementById('textoResultado');
textoResultado.textContent = 'Tamanho: Duplo';
```

código

o que significa

textoResultado	a variável que já guarda o elemento <code><p></code> , encontrado na linha anterior.
.textContent	propriedade que representa o texto que fica dentro daquele elemento.
= 'Tamanho: Duplo'	troca todo o texto atual do elemento por esse novo texto. Quem via "Escolha uma opção" agora vê "Tamanho: Duplo".

`textContent` funciona nos dois sentidos. Você pode **escrever** nele (como no exemplo acima) para mudar o que aparece na tela, e pode **ler** ele para saber o que está escrito ali agora:

```
console.log(textoResultado.textContent); // imprime o texto atual do elemento
```

TEXTO FIXO + VALOR QUE MUDA

Na prática, quase sempre você vai montar o novo texto grudando uma parte fixa com um valor calculado, usando o `+` da seção 04:

```
textoResultado.textContent = 'Total: R$ ' + total.toFixed(2);
```

Exemplos nos três temas

```
// lanche
textoTamanho.textContent = 'Tamanho: ' + tamanhoEscolhido;

// roupa
textoResumo.textContent = 'Camiseta ' + tamanhoEscolhido + ', cor ' +
corEscolhida;

// viagem
textoPreco.textContent = 'Total: R$ ' + precoPassagem.toFixed(2);
```

Reagindo a eventos: addEventListener

É isso que transforma uma página parada em algo interativo: código que só roda quando algo acontece.

Até aqui, todo código deste guia roda uma vez, assim que a página carrega. Mas o objetivo é reagir a ações do usuário: um clique, uma escolha marcada, um campo alterado. Para isso existe `addEventListener`, que quer dizer literalmente "escutar um evento".

index.html

```
<button id="botaoDuplo">Duplo</button>
```

script.js

```
const botaoDuplo = document.getElementById('botaoDuplo');

botaoDuplo.addEventListener('click', mudarParaDuplo);

function mudarParaDuplo() {
  textoTamanho.textContent = 'Tamanho: Duplo';
}
```

código	o que significa
botaoDuplo	o elemento que vai ser observado, encontrado antes com <code>getElementById</code> .
<code>.addEventListener(...)</code>	método que diz "quando algo acontecer com este elemento, rode uma função".
<code>'click'</code>	o tipo de evento esperado: um clique.
mudarParaDuplo	o nome da função que deve rodar quando o clique acontecer. Repare: sem parênteses.
<code>function mudarParaDuplo() { ... }</code>	a função em si, definida separadamente. O que está entre chaves só roda quando o clique acontece, nunca antes.

SEM PARÊNTESES

Em `addEventListener('click', mudarParaDuplo)` o nome da função vai **sem parênteses**. Se você escrever `mudarParaDuplo()` (com parênteses), o JavaScript executa a função imediatamente, na hora que a página carrega, em vez de esperar o clique. Isso costuma causar um bug silencioso: o código não dá erro, mas o texto muda sozinho antes de qualquer clique, e depois o clique não faz mais nada.

click e change: dois tipos de evento diferentes

'click' é para elementos que se clica, como botões. Para campos de formulário (radio, checkbox, select) o evento mais usado é 'change', que dispara quando o valor ou a marcação daquele campo muda:

```
tamanhoGrande.addEventListener('change', atualizarTela);
extraBacon.addEventListener('change', atualizarTela);
```

Repare que as duas linhas apontam para a **mesma função**, `atualizarTela`. Isso é comum e é uma boa prática: em vez de escrever uma função diferente para cada input, você escreve uma função só que recalcula tudo, e liga todos os inputs relevantes nela.

Exemplos nos três temas

```
// lanche
tamanhoDuplo.addEventListener('change', atualizarTela);

// roupa
tamanhoM.addEventListener('change', atualizarTela);
estampaPersonalizada.addEventListener('change', atualizarTela);

// viagem
classeExecutiva.addEventListener('change', atualizarTela);
extraSeguro.addEventListener('change', atualizarTela);
```

TESTANDO SE O EVENTO ESTÁ FUNCIONANDO

Antes de escrever a lógica completa, teste se o evento está mesmo disparando com um `console.log` simples:

```
extraBacon.addEventListener('change', function () {
  console.log('mudou!');
});
```

Se nada aparecer no Console ao marcar a caixinha, o problema está no id, no evento ou em algum erro anterior no arquivo, não na lógica que você ainda vai escrever.

Lendo formulários: value, checked e Number()

Como descobrir o que o usuário escolheu.

value: o valor de um campo

Todo elemento de formulário (radio, checkbox, input de texto) tem uma propriedade `.value` que guarda o que está escrito no atributo `value` do HTML:

index.html

```
<input id="tamanhoGrande" type="radio" name="tamanho" value="Grande">
```

script.js

```
console.log(tamanhoGrande.value); // "Grande"
```

value SEMPRE VEM COMO TEXTO

Mesmo que o value pareça um número no HTML (`value="15"`), `.value` devolve uma **string**, nunca um number. Isso importa porque strings não somam, elas grudam (seção 04). É para resolver isso que existe `Number()`, explicado mais abaixo.

checked: está marcado ou não

Radio buttons e checkboxes têm uma propriedade a mais, `.checked`, que é um **boolean**: `true` se estiver marcado, `false` se não estiver.

```
console.log(extraBacon.checked); // true ou false
```

Na prática, `.checked` é o que você usa dentro de um `if` para decidir o que fazer (a seção 09 detalha isso):

```
if (extraBacon.checked) {  
  total = total + 4;  
}
```

Number(): transformando texto em número de verdade

Quando você precisa **calcular** com um valor que veio de um campo (por exemplo, um input onde o usuário digita a própria quantidade), esse valor chega como string e precisa ser convertido antes de entrar numa conta:

```
const quantidadeDigitada = document.getElementById('quantidade').value; //  
string, ex: "3"  
const quantidadeNumero = Number(quantidadeDigitada); //  
number, 3  
  
let totalComQuantidade = precoBase * quantidadeNumero; // agora funciona como  
conta
```

código

o que significa

Number(...)	função pronta do JavaScript que tenta converter o que está entre parênteses para o tipo number.
Number("3")	vira o número 3, sem aspas.
Number("três")	vira NaN ("Not a Number"), porque "três" escrito por extenso não é um número válido para o JavaScript converter.

QUANDO O Number() NÃO É NECESSÁRIO

Se você só está comparando o value com um texto (por exemplo, `if (corEscolhida.value === "Azul")`), não precisa de Number(), porque os dois lados são strings. Number() só entra quando o valor vai ser usado numa **conta** (soma, multiplicação) e não numa comparação de texto.

Decisões: if, else if e else

Como o código escolhe um caminho diferente dependendo da situação.

Um `if` testa uma condição. Se a condição for verdadeira, o bloco de código entre chaves roda. Se for falsa, esse bloco é simplesmente ignorado.

```
if (extraBacon.checked) {  
  total = total + 4;  
}
```

código	o que significa
<code>if (...)</code>	"se a condição entre parênteses for verdadeira, faça o que vem a seguir".
<code>extraBacon.checked</code>	a condição em si. Já é um boolean (true ou false), então não precisa de <code>=== true</code> .
<code>{ ... }</code>	o bloco que só roda se a condição for verdadeira. Tudo entre essas chaves.

else: o caminho alternativo

`else` roda quando a condição do `if` é falsa. Junto, os dois cobrem os dois únicos caminhos possíveis:

```
if (extraBacon.checked) {  
  textoBacon.textContent = 'Com bacon';  
} else {  
  textoBacon.textContent = 'Sem bacon';  
}
```

else if: quando há mais de duas opções

Quando existem três ou mais opções que se **excluem** (só uma pode valer por vez, como um radio button), usa-se uma cadeia de `else if`:

```

let precoBase = 0;

if (tamanhoSimples.checked) {
  precoBase = 15;
} else if (tamanhoDuplo.checked) {
  precoBase = 20;
} else if (tamanhoTriplo.checked) {
  precoBase = 25;
} else {
  precoBase = 0;
}

```

código

o que significa

<code>if (tamanhoSimples.checked)</code>	primeira pergunta: Simples está marcado? Se sim, precoBase vira 15 e o resto da cadeia é ignorado.
<code>} else if (tamanhoDuplo.checked)</code>	só é testado se a pergunta anterior foi falsa. Se Duplo estiver marcado, precoBase vira 20.
<code>} else if (tamanhoTriplo.checked)</code>	só é testado se as duas anteriores foram falsas. Se Triplo estiver marcado, precoBase vira 25.
<code>} else { precoBase = 0; }</code>	roda se nenhuma das opções acima estiver marcada, cobrindo o caso de ninguém ter escolhido nada ainda.

if/else if PARA ESCOLHA ÚNICA, VÁRIOS if PARA EXTRAS

Uma dúvida comum: por que não usar `else if` para os adicionais (bacon, ovo, queijo)? Porque radio button é exclusivo (só um pode estar marcado, então faz sentido perguntar em cadeia), mas checkbox é independente (o usuário pode marcar bacon e ovo ao mesmo tempo). Se você encadear os adicionais com else if, o código para de checar os outros assim que um for verdadeiro, e um adicional marcado é ignorado. A forma certa é um if separado para cada checkbox:

```

if (extraBacon.checked) { total = total + 4; }
if (extraOvo.checked) { total = total + 3; }
if (extraQueijo.checked) { total = total + 5; }

```

Exemplos nos três temas

```
// roupa: escolha exclusiva de tamanho
if (tamanhoP.checked) {
    precoBase = 39;
} else if (tamanhoM.checked) {
    precoBase = 39;
} else if (tamanhoG.checked) {
    precoBase = 45;
}

// viagem: escolha exclusiva de classe
if (classeEconomica.checked) {
    precoPassagem = 350;
} else if (classeExecutiva.checked) {
    precoPassagem = 900;
} else if (classePrimeira.checked) {
    precoPassagem = 1800;
}
```

Funções: organizando o código

Uma função é um pedaço de código com nome, que você pode rodar quantas vezes quiser sem reescrevê-lo.

Criando e chamando uma função

```
function calcularTotal() {  
  let total = 0;  
  total = total + 15;  
  return total;  
}  
  
const resultado = calcularTotal();  
console.log(resultado); // 15
```

código

o que significa

`function calcularTotal()`

cria uma função chamada `calcularTotal`. Os parênteses vazios significam que ela não recebe nenhuma informação de fora.

`{ ... }`

o corpo da função: o código que roda toda vez que ela é chamada.

`return total;`

devolve o valor de `total` para quem chamou a função. Sem `return`, a função roda mas não entrega nada de volta.

`calcularTotal()`

chama a função (agora com parênteses, porque aqui você quer que ela rode). O valor devolvido é guardado em `resultado`.

DEFINIR NÃO É CHAMAR

Escrever `function calcularTotal() { ... }` só ensina ao JavaScript o que a função faz, não roda o código dentro dela. O bloco só executa quando alguém escreve `calcularTotal()`, com parênteses, em algum outro lugar do arquivo (ou quando um evento chama ela, como na seção 07).

Por que separar calcular de mostrar

Um padrão usado no curso inteiro: uma função **calcula** um valor e devolve com `return`, outra função **mostra** esse valor na tela com `textContent`. São responsabilidades diferentes.

```
function calcularTotal() {
  let total = 0;

  if (tamanhoSimples.checked) {
    total = total + 15;
  } else if (tamanhoDuplo.checked) {
    total = total + 20;
  }

  if (extraBacon.checked) {
    total = total + 4;
  }

  return total;
}

function atualizarTela() {
  const total = calcularTotal();
  textoTotal.textContent = 'Total: R$ ' + total.toFixed(2);
}
```

código

o que significa

<code>function atualizarTela()</code>	a função que junta tudo: pede o cálculo pronto e escreve na tela.
<code>const total = calcularTotal();</code>	chama a outra função, pega o valor que ela devolveu e guarda numa variável local chamada total.
<code>textoTotal.textContent = ...</code>	usa esse valor para montar e mostrar a frase final.

POR QUE ISSO AJUDA

Com essa separação, todo evento (mudar tamanho, marcar um extra) chama a mesma função `atualizarTela`. Ela sempre recalcula do zero e escreve o resultado atualizado. Você não precisa lembrar de atualizar o texto manualmente em cada lugar do código, então fica muito mais difícil esquecer de atualizar alguma coisa.

11 · PROJETO

Tudo junto: um projeto completo, linha por linha

O exemplo do lanche, do início ao fim, com cada linha explicada.

index.html (trecho relevante)

```
<input id="simples" type="radio" name="tamanho" value="Simples"> Simples  
<input id="duplo" type="radio" name="tamanho" value="Duplo"> Duplo  
<input id="bacon" type="checkbox" value="Bacon"> Bacon extra  
<p id="textoTotal">Total: R$ 0.00</p>
```

script.js (arquivo completo)


```
// 1. ELEMENTOS DO HTML
const simples = document.getElementById('simples');
const duplo = document.getElementById('duplo');
const bacon = document.getElementById('bacon');
const textoTotal = document.getElementById('textoTotal');

// 2. FUNÇÕES
function calcularTotal() {
    let total = 0;

    if (simples.checked) {
        total = total + 15;
    } else if (duplo.checked) {
        total = total + 20;
    }

    if (bacon.checked) {
        total = total + 4;
    }

    return total;
}

function atualizarTela() {
    const total = calcularTotal();
    textoTotal.textContent = 'Total: R$ ' + total.toFixed(2);
}

// 3. EVENTOS
simples.addEventListener('change', atualizarTela);
duplo.addEventListener('change', atualizarTela);
bacon.addEventListener('change', atualizarTela);

// 4. PRIMEIRA EXIBIÇÃO
atualizarTela();
```

linha	o que acontece
<code>const simples = document.getElementById('simples');</code>	acha o radio button "Simples" na página e guarda numa variável. O mesmo para duplo, bacon e textoTotal logo abaixo.
<code>function calcularTotal() {</code>	começa a definir a função que calcula o preço. Nada aqui roda ainda, é só a receita.
<code>let total = 0;</code>	toda vez que a função roda, o cálculo começa do zero. Isso evita que valores de escolhas antigas fiquem "grudados".
<code>if (simples.checked) { ... } else if (duplo.checked) { ... }</code>	verifica qual tamanho está marcado e soma o preço correspondente a total. Só um dos dois blocos roda.
<code>if (bacon.checked) { ... }</code>	verificação separada, porque bacon é independente do tamanho escolhido.
<code>return total;</code>	devolve o valor final calculado para quem chamou a função.
<code>function atualizarTela() {</code>	começa a segunda função, responsável por mostrar o resultado na tela.
<code>const total = calcularTotal();</code>	chama a primeira função e guarda o resultado numa variável local.
<code>textoTotal.textContent = 'Total: R\$ ' + total.toFixed(2);</code>	escreve o valor formatado no parágrafo da página.
<code>simples.addEventListener('change', atualizarTela);</code>	liga o evento: sempre que o usuário mudar essa escolha, atualizarTela roda de novo. O mesmo para duplo e bacon.
<code>atualizarTela();</code>	chamada única, fora de qualquer evento, para que a tela já mostre "Total: R\$ 0.00" corretamente assim que a página abre, antes de qualquer clique.

ESSE ARQUIVO É UM MODELO, NÃO UMA FÓRMULA FIXA

A estrutura em quatro blocos (elementos, funções, eventos, primeira exibição) funciona para qualquer tema. Para adaptar para roupa ou viagem, troque os ids pelos do seu HTML, troque os valores dentro do cálculo e troque o texto que aparece no resumo. A lógica por trás continua exatamente a mesma.

Erros comuns e como ler o Console

Quando algo quebra, o Console quase sempre diz o suficiente para descobrir onde.

sintoma	onde olhar primeiro
Cannot set properties of null	o id usado no getElementById não existe no HTML, ou está escrito diferente (maiúscula, espaço).
total aparece como NaN	algum valor usado na conta ainda é string. Provavelmente falta um Number() em algum .value.
o total gruda os números em vez de somar	mesmo problema: um dos dois lados do + é string, não number.
a tela não muda quando clico ou marco algo	falta addEventListener, ou o nome da função foi escrito com parênteses (chamando na hora, em vez de passar a referência).
o texto muda sozinho ao abrir a página, antes de qualquer clique	a função foi chamada com parênteses dentro do addEventListener, então rodou na hora em vez de esperar o evento.
só uma checkbox conta no cálculo	os extras foram encadeados com else if em vez de vários if separados.
is not a function	nome de método escrito errado (getElementById em vez de getElementByld, por exemplo). JavaScript diferencia maiúsculas de minúsculas em tudo.
Unexpected token	faltou fechar uma chave, um parêntese ou uma aspas em algum lugar acima da linha apontada pelo erro.

COMO LER UMA MENSAGEM DE ERRO

O Console geralmente mostra o tipo de erro, uma descrição curta e o número da linha onde ele foi detectado. A linha apontada nem sempre é onde está o problema de verdade (às vezes é só onde o JavaScript desistiu de tentar entender), mas é sempre um bom ponto de partida. Leia a mensagem em voz alta, boa parte delas descreve o problema em inglês simples.

Glossário rápido

Os termos deste guia, em ordem alfabética.

boolean

tipo de dado que só pode valer true ou false.

checked

propriedade de radio buttons e checkboxes que diz se estão marcados (true ou false).

const

palavra usada para criar uma variável que não muda de valor depois de criada.

console.log

comando que imprime um valor no Console do navegador, usado para testar e depurar código.

DOM

Document Object Model, a forma como o JavaScript enxerga a estrutura da página HTML.

evento

algo que acontece na página, como um clique ou uma mudança de valor, que o JavaScript pode escutar e reagir.

função

bloco de código nomeado que pode ser executado (chamado) quantas vezes for necessário.

getElementById

método que procura, dentro de toda a página, o elemento HTML com um id específico.

let

palavra usada para criar uma variável que pode receber um novo valor depois de criada.

NaN

"Not a Number", valor especial que aparece quando uma conversão para número falha.

Number()

função pronta que tenta converter um valor (geralmente uma string) para o tipo number.

parâmetro

informação que uma função recebe entre parênteses para trabalhar com ela.

return

palavra usada dentro de uma função para devolver um valor para quem a chamou.

string

tipo de dado que representa texto, sempre escrito entre aspas.

textContent

propriedade que representa o texto dentro de um elemento HTML, pode ser lida ou alterada.

value

propriedade que representa o valor atual de um campo de formulário, sempre devolvido como string.

Para continuar

- ☐ Rerler este guia com o navegador aberto e testar cada exemplo digitando (não copiando).
- ☐ Rodar o exemplo completo da seção 11 do zero, sem consultar, e comparar com o original.
- ☐ Adaptar o exemplo completo para o tema do seu próprio projeto.
- ☐ Voltar aqui sempre que uma mensagem de erro não fizer sentido de primeira.

PRÓXIMO PASSO

O material do Dia 2 (slides) parte exatamente daqui e aplica esses mesmos conceitos direto no seu projeto: leitura de escolhas, cálculo de um resultado e atualização da tela. Se algo por lá não fizer sentido, a explicação detalhada está neste guia.